Technische Universität Wien

Algorithmics

course material: slides

Ostrich algorithm

14 languages

Article Talk Read Edit View history

From Wikipedia, the free encyclopedia

In computer science, the **ostrich algorithm** is a strategy of ignoring potential problems on the basis that they may be exceedingly rare. It is named after the ostrich effect which is defined as "to stick one's head in the sand and pretend there is no problem". It is used when it appears the situation may be more cost-effectively managed by allowing the problem to continue to occur rather than to attempt its prevention.

aglaia norza

November 1, 2026

norzaglaia@gmail.com

github.com/AglaiaNorza

Contents

1 Computational Complexity (Cheat Sheet) 2
- 1.1 P and NP . . . 2
- 1.2 Space Classes (L and NL) . . . 4
- 1.3 The polynomial hierarchy . . . 6
  - 1.3.1 The Hierarchy Ladder . . . 7
  - 1.3.2 The Quantifier Form and Absorption Rule . . . 7
  - 1.3.3 The class DP . . . 8
- 1.4 The Class PSPACE . . . 9
- 1.5 The Exponential Classes . . . 10

2 A* Algorithm 12
- 2.1 Informed Search . . . 12
  - 2.1.1 Greedy Best First Search . . . 12
- 2.2 The A* Algorithm . . . 12
- 2.3 Properties of A* . . . 13
  - 2.3.1 Heuristics and Optimality . . . 14
- 2.4 Complexity and Variants . . . 15
  - 2.4.1 Improvements and Variants . . . 15

3 Planar Graphs 16
- 3.1 Basic Concept . . . 16
- 3.2 Euler's Formula . . . 16
- 3.3 Characterizations of Planar Graphs . . . 16
- 3.4 Linear Planarity Testing Algorithms . . . 18

CHAPTER 1

Computational Complexity (Cheat Sheet)

1.1 | P and NP

Def. 1 The class NP

There are two ways to define the complexity class NP:

1 | NP is the set of decision problems *solvable* in polynomial time by a *nondeterministic* Turing machine

2 | NP is the set of decision problems *verifiable* in polynomial time by a *deterministic* Turing machine:

- that is, $L \in$ NP if there are a polynomial p and a polynomial-time verifier V such that for every x:

$$x \in L \iff \exists u \text{ with } |u| \leq p(|x|) \text{ s.t. } V(x,u) = \text{accept}$$

Showing $L \in$ NP

1 | define the certificate u (the proposed solution)

2 | show that $|u| \leq$ a polynomial in the input length $|x|$ (NB: remember to count bits - n vertices take up $n \log n$ bits, not n)

3 | explain the mechanism of the verifier $V(x,u)$

4 | confirm that the running time of the verifier is polynomial in $|x|$

5 | prove both directions of correctness:

- explain why the verifier accepts a YES-instance
- explain why the verifier rejects a NO-instance (it cannot loop !)
  or, assume that the verifier accepts some certificate u, and prove that x must be a YES-instance

Def. 2 Karp Reduction

A polynomial-time Karp reduction translates an instance of problem A into an instance of problem B. Upper bounds transfer from B to A, meaning if $B \in P$, then $A \in P$. Conversely, hardness transfers from A to B (if $A \leq_p B$ and $A \notin P$, $B \notin P$).

Reductions are also transitive, meaning if $A \leq_p B$ and $B \leq_p C$, then $A \leq_p C$.

We have that $A \leq_p B$ iff there is a polynomial function f such that, for every string x:

$$x \in A \iff f(x) \in B$$

Showing $A \leq_p B$

1 | define the function $f(\text{instance of } A) = \text{instance of } B$, which must be computed from the instance alone

2 | state the size of $f(x)$ and the cost of computing it, in terms of $|x|$

3 | correctness ($\Longrightarrow$): prove that if x is a YES-instance of A, then $f(x)$ is a YES-instance of B (typically involves constructing a solution for $f(x)$ from a solution of x)

4 | correctness ($\Longleftarrow$): prove that if $f(x)$ is a YES-instance of B, then x is a YES-instance of A (typically involves recovering a solution for x from a solution of $f(x)$)

or, do the contrapositive: if x is a NO-instance of A, then $f(x)$ is a NO-instance of B

The core reductions covered in the course are:

- **Logic to logic:** SAT $\leq_p$ NAE-SAT
  - *Reduction by symmetry breaking:* pick a fresh variable s and append it to every clause: $(\ell_1 \vee \ldots \vee \ell_r) \mapsto (\ell_1, \ldots, \ell_r, s)$.
    - NAE has flip symmetry: if β satisfies, flipping all values β' also satisfies.
    - the shared s acts as a reference anchor: without loss of generality, take the solution where $s =$ false.
    - since s guarantees the required false in every clause, the clause is NAE-satisfied if and only if at least one original literal ℓ_i is true (which is the standard SAT condition)
- **logic to graphs:** 3-SAT $\leq_p$ CLIQUE
  - for a formula with m clauses:
    - create 3 vertices per clause (one per literal occurrence), put an edge between two vertices iff they are in different clauses and consistent ($u \neq \neg v$), set target clique size $k = m$
    - since there are no edges within the same clause, a clique of size m must pick exactly one literal per clause; because edges only connect non-contradictory literals, picking an m-clique never selects both x_i and $\neg x_i$, directly defining a valid satisfying truth assignment
- **graph to graph:** CLIQUE $\leq_p$ INDEPENDENT SET $\leq_p$ VERTEX COVER
  - CLIQUE $\leq_p$ IS:
    - *map:* $(G, k) \mapsto (\overline{G}, k)$, where $\overline{G}$ is the complement graph
    - a subset $S \subseteq V$ forms a clique in G (all pairs connected) $\iff$ S is an independent set in $\overline{G}$ (no pairs connected)
  - IS $\leq_p$ VC:
    - *map:* $(G, k) \mapsto (G, |V| - k)$ (flip the target size)
    - S is an independent set $\iff$ $V \setminus S$ is a vertex cover: since no edge can have both endpoints in S, every edge must touch at least one vertex in $V \setminus S$; thus, an IS of size $\geq k$ exists $\iff$ a VC of size $\leq |V| - k$ exists

- **Logic to Coloring:** NAE-3-SAT $\leq_p$ 3-COLORING
  - *Construction / Gadgets:*
    - have a neutral vertex B (assigned color *base*; remaining two colors represent *true* and *false*).
    - for each variable x, connect x and $\neg x$ to each other and both to B (forming a triangle with B); for each clause (ℓ_1, ℓ_2, ℓ_3), create a fresh triangle u_1, u_2, u_3 and add edges (u_1, ℓ_1), (u_2, ℓ_2), and (u_3, ℓ_3)
    - connecting every literal to B forces all literals to take colors from {true, false}, with x and $\neg x$ having opposite colors.
    - the triangle $\{u_1, u_2, u_3\}$ requires all 3 colors.
      - if ℓ_1, ℓ_2, ℓ_3 are all equal (all true or all false), that color is blocked for all u_i, leaving only 2 available colors for a 3-cycle (impossible to 3-color).
      - if ℓ_1, ℓ_2, ℓ_3 are not all equal, the three colors {true, false, B} can always be validly assigned to $\{u_1, u_2, u_3\}$.

Def. 3 **Gadgets**

Often, pieces of the target problem (e.g., a subgraph) can be designed to simulate a specific component of the source problem (e.g., a variable or a clause)

1 | **choice gadget:** how do you represent a boolean variable making a choice (True/False)?
- *example:* in mapping to 3-COLORING, a variable x is represented by a pair of adjacent vertices (x and $\neg x$) that are both connected to a universal "Base" vertex; the Base takes one color, forcing x and $\neg x$ to take the remaining two colors (representing True and False)

2 | **constraint gadget:** how do you enforce the rules of the source problem (e.g., "at least one literal must be true")?
- *example:* build a small subgraph connecting the literal vertices of a clause to a target vertex; the subgraph's edges are designed so that the whole gadget is only valid (e.g., fully colorable) if the inputs satisfy the source problem's specific rule

1.2 | Space Classes (L and NL)

Def. 4 **The Classes L and NL**

Instead of bounding time, we bound the memory (space) used on the work tape. The input tape is read-only, and we only count the memory used on the read/write work tape.

- **L (log-space):** the set of decision problems solvable by a *deterministic* Turing machine using $O(\log |x|)$ space
- **NL (nondeterministic log-space):** the set of decision problems solvable by a *nondeterministic* Turing machine using $O(\log |x|)$ space

Savitch / Immerman-Szelepcsényi: NL = coNL.

Showing $L \in$ L or NL

To show a problem is in L or NL, define an algorithm and prove it uses logarithmic space

1 | write the algorithm as a numbered list.

2 | list exactly what the work tape needs to hold (e.g., counters, vertex indices, pointers).

3 | write out the exact bit size for each variable, and show that the sum of all bit sizes is bounded by $O(\log |x|)$

- *Note:* a vertex index out of n vertices takes $\lceil \log_2 n \rceil$ bits; a counter up to n takes $\lceil \log_2 n \rceil$ bits

Def. 5 **Log-space Reductions ($A \leq_L B$)**

A log-space reduction from A to B is a deterministic algorithm that computes a mapping function f, transforming an instance of A into an instance of B, using only $O(\log |x|)$ work-tape space.

Showing $A \leq_L B$

1 | define f(instance of A) = instance of B, computed deterministically.

2 | state the variables needed to compute the map and their bit sizes, showing the sum is $O(\log |x|)$. (Do not count the space used to write the output, only the work tape).

3 | correctness ($\Longrightarrow$): prove that if x is a YES-instance of A, then $f(x)$ is a YES-instance of B

4 | crrectness ($\Longleftarrow$): prove that if $f(x)$ is a YES-instance of B, then x is a YES-instance of A (or prove the contrapositive for NO-instances)

Problems in L

- **BALANCED PARENTHESES:** given a string over '(' and ')', are they properly balanced? (uses a counter to track the current depth).
- **PALINDROME:** given a string, is it equal to its reverse? (uses two pointers)
- **EVEN-DEGREE:** given an undirected graph, does every vertex have an even degree? (uses a counter to iterate over edges for each vertex)
- **UNDIRECTED PATH:** given an undirected graph and vertices s, t, is there a path from s to t? (the directed version is NL-complete !)

Problems in NL

- **PATH (NL-Complete):** given a directed graph and vertices s, t, is there a directed path from s to t? *(canonical NL-complete problem; NL-hardness proofs start by reducing from PATH, often using configuration graphs)*
- **STRONG-CONNECTIVITY:** given a directed graph, is there a directed path between every pair of vertices?

- **CYCLE-THROUGH:** given a directed graph and a vertex v, is there a directed cycle containing v?
- **2-SAT and 2-UNSAT:** is a given 2-CNF formula satisfiable/unsatisfiable? (building an implication graph and searching for a directed path from a literal x to $\neg x$ and vice versa)

1.3 | The polynomial hierarchy

The polynomial hierarchy (PH) generalizes the complexity classes P, NP and coNP by classifying computational problems based on the number of alternating quantifiers required to verify their solutions.

(In standard NP, a non-deterministic machine guesses one certificate and checks it in polynomial time. The polynomial hierarchy extends this by allowing multiple, alternating phases of guessing. Each phase represents a shift in the logical burden of proof - from "does there exist?" (∃) to "is it true for all?" (∀). The number of times this burden shifts dictates the problem's level in the hierarchy.)

The base of every problem in the polynomial hierarchy is the polynomial-time predicate R

Def. 6 **Polynomial-time predicate**

A **polynomial-time predicate** $R(x, u_1, \dots, u_k)$ is a condition on strings that a polynomial-time algorithm checks.

- u_i are the certificates (guesses or moves generated by the quantifiers)
- the length of every u_i needs to be bounded by some polynomial relative to the size of the input x ($|u_i| \leq p(|x|)$)
- R then evaluates the conditions and outputs true/false in polynomial time

The classes of PH depend on the quantifier structure of the problem:

Def. 7 **Quantifier block**

A block is a *maximal run of equal quantifiers*. Two quantifiers of the same kind in a row are one block, since a pair of strings is one string; the *quantifier level* counts blocks.

The class Σ_k

A language L belongs in the class Σ_k if its defining logical statement starts with an existential quantifier, and has exactly $k-1$ alternations.

- e.g. $x \in L \in \Sigma_3 \iff \exists u_1 \forall u_2 \exists u_3 : R(x, u_1, u_2, u_3)$.

- We have that $\Sigma_1 = \text{NP}$!

The class Π_k

A language L belongs in the class Π_k if its defining logical statement starts with a universal quantifier, and has exactly $k-1$ alternations.

- e.g. $x \in L \in \Pi_2 \iff \forall u_1 \exists u_2 : R(x, u_1, u_2)$.

Π_k is the complement of Σ_k): $\Pi_i = \{L : \bar{L} \in \Sigma_i\}$ (negating the satatement swaps every ∃ with a

∀)

- We have that $\Pi_1 = \text{coNP}$!

1.3.1 | The Hierarchy Ladder

We have that:

$$\Sigma_i \cup \Pi_i \subseteq \Sigma_{i+1} \cap \Pi_{i+1}$$

, which means that any language in Σ_i or Π_i is also contained within both classes of the next level up (Σ_{i+1} and Π_{i+1}). This is proven by adding a "dummy" quantifier.

If a language is defined by a polynomial-time predicate R, we can artificially introduce a new quantifier over a variable v that R completely ignores. Because the verifier never evaluates v, the truth value of the statement is unchanged, but the formula gains the extra alternation required to place it in a higher class.

- moving from Σ_1 to Σ_2 (∃∀):

  suppose $L \in \Sigma_1$ (NP), defined as $x \in L \iff \exists u : R(x,u)$.

  we can rewrite this as $x \in L \iff \exists u \forall v : R'(x,u,v)$, where R' executes $R(x,u)$ and ignores v - the problem is now technically in Σ_2

- moving from Σ_1 to Π_2 (∀∃):

  using the same $L \in \Sigma_1$, we can alternatively pad the front of the statement: $x \in L \iff \forall v \exists u : R'(x,u,v)$, where R' again ignores v - the problem is now technically in Π_2.

Through this padding mechanism, the lower classes are subsumed by the intersection of the higher classes.

1.3.2 | The Quantifier Form and Absorption Rule

The Quantifier Form

To define a language L using the quantifier form, it is necessary to provide the statement, sizes, predicate, and class:

1 | **statement**: explicitly state $x \in L \iff Q_1 u$ (what u is, its size) $Q_2 v$ (what v is, its size): $R(x,u,v)$, written in words (e.g., "there is a ...such that for every ...")

2 | **sizes**: state that each object is bounded by an explicit polynomial in $|x|$
(N.B: a condition that is part of the question, e.g. "the set has at least k vertices", is checked inside R; it is not a size bound.)

3 | **predicate**: state that R is decidable in polynomial time, providing the reason

4 | **class**: read off the prefix: ∃ → NP, ∀ → coNP, ∃∀ → Σ_2, ∀∃ → Π_2

Not every ∀ or ∃ statement creates a new hierarchy level:

The absorb rule

The absorb rule dictates that if the candidate space for a quantifier is small enough that it can be listed and checked in polynomial time (e.g., checking every node in a graph), that quantifier

is "absorbed" directly into the polynomial-time predicate R (and it is called a **loop**).

A quantifier only remains in the prefix - and thus pushes the problem to a higher complexity class - if it iterates over an exponentially large candidate space, e.g. all possible 2^n Boolean truth assignments

Example: INDEPENDENT SET. Consider the problem: *is there a set $S \subseteq V$ with $|S| \geq k$, such that there are no edges inside S?*

The logical statement is: *there exists* ($\exists$) a set $S \subseteq V$ with $|S| \geq k$ such that *for every pair* ($\forall$) $\{a, b\} \subseteq S$, $ab \notin E$.

- The $\exists$ block remains: thge candidate space for S is 2^n (all possible subsets of n vertices). Because this scales exponentially, a deterministic machine cannot check all possibilities in polynomial time: the quantifier block remains.
- The $\forall$ block is absorbed: once a specific candidate set S is handed to the verifier, the inner condition checks "for every pair". The maximum number of vertex pairs in S is bounded by n^2. Therefore, the machine simply runs a nested loop over the pairs to check the adjacency list. The $\forall$ quantifier is **absorbed** into the deterministic polynomial-time predicate R.

Therefore, the problem is in $\Sigma_1 = \text{NP}$.

To understand complete problems for the polynomial hierarchy, we use Quantified Boolean Formulas, which are the natural extension of the standard Boolean satisfiability problem (SAT).

Def. 8

A quantified Boolean formula is typically written in prenex form (meaning all quantifiers are pulled to the front) followed by a quantifier-free Boolean formula called the matrix (ψ).

$$Q_1 x_1 Q_2 x_2 \ldots Q_q x_q [\psi]$$

- where each Q_i is a quantifier ($\exists$ or $\forall$)

The formula is fully quantified, meaning every variable in the matrix ψ is bound by a quantifier (so the entire formula is either true or false).

Just as SAT is complete for NP, specific formats of QBFs act as the canonical complete problems for every level of the hierarchy.

- Σ_2-SAT: "given a formula $\exists X \forall Y[\psi]$, is it true?" is a complete problem for Σ_2.

  the matrix ψ must be an unrestricted Boolean formula: if ψ were in CNF, the $\forall Y$ check could be evaluated clause by clause, and the complexity would drop down to NP
- Π_2-SAT: "given a formula $\forall X \exists Y[\psi]$, is it true?" is Π_2-complete, and it remains complete even if ψ is in CNF

1.3.3 | The class DP

The class DP sits just above NP and coNP, but below the second level of the polynomial hierarchy ($\Sigma_2 \cap \Pi_2$). It captures problems that require solving an NP question and a coNP question simultaneously.

Def. 9 **DP**

A language belongs to DP if it is the intersection of two languages over the same instances: one from NP, and one from coNP:

$$\text{DP} = \{L_1 \cap L_2 : L_1 \in \text{NP}, L_2 \in \text{coNP}\}$$

N.B.: the class DP is different from NP ∩ coNP, which represents the class of single languages that have both a fast yes and a fast no verifier

- actually, DP = NP ∩ coNP if and only if NP = coNP

Showing $L \in$ DP

To show that a language L is in DP, you must split its defining condition into two independent halves.

For example: "exactly one solution" becomes "there is one" and "there are no two". "The optimum is exactly k" becomes "at least k" and "not at least $k+1$" (minimizing: "at most k" and "not at most $k-1$").

1 | **define the two halves**: explicitly define language L_1 (the "at least" or existence condition) and language L_2 (the "not more than" or uniqueness condition)

2 | **show the intersection**: formally state that a string $x \in L$ if and only if $x \in L_1 \cap L_2$

3 | **prove $L_1 \in$ NP**: define the certificate for L_1 (e.g., a solution of size k) and state that it is polynomially bounded and can be verified in polynomial time

4 | **prove $L_2 \in$ coNP** (usually by proving that $\overline{L_2}$ is in NP);

1.4 | The Class PSPACE

Def. 10 **PSPACE**

PSPACE is the class of decision problems solvable by a deterministic Turing machine using a polynomial amount of memory space.

Formally, it is defined as PSPACE = $\bigcup_{k\geq 1}$ SPACE($|x|^k$).

A fundamental property of space complexity (Savitch's Theorem) is that PSPACE = NPSPACE.

- While PH restricts problems to a constant, fixed number of quantifier alternations, PSPACE allows for an unbounded number of alternations (up to a polynomial limit). Consequently, the entire Polynomial Hierarchy is contained within PSPACE (PH ⊆ PSPACE).
- PSPACE captures the complexity of adversarial two-player games: any two-player game with board positions of polynomial size and a polynomially bounded maximum number of moves belongs in PSPACE. The nested alternating quantifiers (∃∀∃∀ ...) model the "if I move here, and you move there..." search required to determine a guaranteed winning strategy

The Complete Problem (TQBF): the problem of determining if a True Quantified Boolean Formula (TQBF) is true is PSPACE-complete. An instance of TQBF is a fully quantified Boolean formula

$Q_1 x_1 \dots Q_q x_q[\psi]$.

- unlike Σ_k-SAT, which is capped at k quantifier blocks, the number of quantifiers q in TQBF can grow linearly with the input size.

TQBF can be evaluated in polynomial space using a recursive algorithm. The algorithm evaluates each quantifier $Q_i x_i$ by testing both possible values of x_i (True and False). Because it can completely erase and reuse memory from the true branch when checking the false branch, the maximum memory required scales linearly with the recursion depth, keeping it in polynomial space.

1.5 | The Exponential Classes

These classes scale time and space bounds to the exponential level, dealing with problems where the input size allows for massive computational resources.

Def. 11 **The classes**

- EXP $= \bigcup_{k\geq 1} \text{TIME}(2^{|x|^k})$
- NEXP $= \bigcup_{k\geq 1} \text{NTIME}(2^{|x|^k})$

  kust as NP uses polynomially bounded certificates, NEXP can be viewed through a certificate lens: a language $L \in$ NEXP if there are certificates of exponential size (up to $2^{|x|^c}$) that can be verified by a deterministic algorithm in time polynomial to the *certificate's* size (which is exponential relative to the original input x)
- EXPSPACE $= \bigcup_{k\geq 1} \text{SPACE}(2^{|x|^k})$

A key concept at the exponential level is how data representation affects complexity. A **succinct** representation compresses an exponentially large structure into a small input.

- **succinct graphs:** instead of listing all edges of an explicit graph, a succinct graph uses a Boolean circuit C, which takes two b-bit vertex IDs and outputs 1 if there is an edge, and 0 otherwise

  because the input C is exponentially smaller than the explicit graph it represents, asking standard questions about the graph causes an exponential blowup in complexity:
  - a problem in P on an explicit graph becomes EXP-complete for a succinct graph
  - a problem in NP on an explicit graph (e.g., Hamiltonian Path) becomes NEXP-complete for a succinct graph

Padding Arguments: Upward Translation Padding is a proof technique demonstrating that a collapse at a lower complexity class triggers a collapse at a higher exponential class. The core mechanism involves artificially inflating the size of an input string to shift the problem into a lower complexity class relative to its new size.

The Padding Mechanism

1 | Consider a language $L \in$ NEXP

2 | Create a new "padded" language L' by appending dummy symbols to each input x until the new length is $N = 2^{|x|^k}$

3 | The original non-deterministic algorithm still takes N steps to evaluate the unpadded prefix x. Because N steps is polynomial relative to the massive new input size N, the padded language L' is structurally in NP

4 | If we assume P $=$ NP, then L' must be in P. To decide the original x, a deterministic machine simply writes out the pad (taking $2^{|x|^k}$ steps) and runs the P algorithm on L' (taking $2^{O(|x|^k)}$ steps). This proves $L \in$ EXP

Through padding, equalities carry upward, but inequalities carry downward via their contrapositive:

- P = NP $\implies$ EXP = NEXP.
- NP = coNP $\implies$ NEXP = coNEXP.
- EXP $\neq$ NEXP $\implies$ P $\neq$ NP

CHAPTER 2

A* Algorithm

2.1 | Informed Search

When trying to find the quickest or shortest route from a source node s to a target node t on a street network, uninformed search algorithms often aren't as fast as they could be.

For example, Dijkstra's algorithm is a classic uninformed search algorithm (that supports non-negative weights) that always examines the closest not-yet-examined node. On a simple grid, Dijkstra's expands uniformly in all directions, corresponding essentially to a Breadth-First-Search (BFS). While accurate, its simple implementation yields a time complexity of $O(|V|^2)$, which is typically too slow for large networks.

To speed up the calculation, we can exploit the properties of the graph by applying heuristic information to evaluate each reached node and always expand the most promising one.

2.1.1 | Greedy Best First Search

A purely heuristic approach evaluates nodes based on a function $h(x)$, which represents the "direct" (unobstructed) distance to the goal t. Common examples include the Euclidean distance $d_2(x, t)$ or the Manhattan distance $d_1(x, t)$.

Greedy Best First Search always expands the reached node with the minimum distance to the goal. While this minimizes the number of expanded nodes, it **does not guarantee an optimal solution**, as it can easily get trapped by obstacles or take suboptimal detours.

2.2 | The A* Algorithm

To avoid the inefficiencies of Dijkstra's algorithm and the mistakes of Greedy Best First Search, the **A* Algorithm** (Hart, Nilsson, Raphael, 1968) combines the advantages of both.

It utilizes a priority function $f(x)$ for all $x \in V$, defined as:

$$f(x) = g(x) + h(x)$$

Where:

- $g(x)$ is the best known cost so far to reach node x from the start node s (the "Dijkstra" part).
- $h(x)$ is the estimated cost from x to the goal t, also known as the "cost-to-go" (the "Greedy" part).

This way, $f(x)$ estimates the **total cost** of the path from s to t that passes through node x.

Implementation and Execution Details

During execution, A* maintains a priority queue of nodes to expand, sorted by their $f(x)$ value:

A* Algorithm

Initialize $g(s) = 0$ and $g(x) = \infty$ for all $x \in V \setminus \{s\}$
Insert source s into priority queue Q with priority $f(s) = h(s)$
while $Q \neq \emptyset$ **do**
    extract node x with minimum $f(x)$ from Q
    if $x = t$ **then**
        return $f(t)$ and the path (using predecessor list)
    end if
    for all outgoing edges $(x, v) \in A$ **do**
        $g' \leftarrow g(x) + w_{xv}$
        if $g' < g(v)$ **then** ▷ found a new or better path to v
            $g(v) \leftarrow g'$
            $pred(v) \leftarrow x$
            insert or update node v in Q with priority $f(v) = g' + h(v)$
        end if
    end for
end while
return no path to target t

NB:

- the algorithm continues until the target node t is *selected for expansion* (extracted from the queue), not until it is seen
- A* may find a shorter path to a node that has already been expanded; in this case, the node gets its $g(x)$ updated and is re-inserted into the priority queue

2.3 | Properties of A*

Thm. 1 **Termination**

A* will always terminate in a limited time.

Proof

The inner loop performs a number of iterations proportional to the out-degree of the node ($O(|V|)$), which terminates. There are finitely many acyclic paths from s to other nodes in the graph, and A* only ever considers acyclic paths. On each major iteration, a new acyclic path is considered because a node is either expanded for the first time, or it is re-expanded via a shorter path. Thus, at worst, A* explores every acyclic path.

Thm. 2 **Completeness**

If a path from s to t exists, A* will always find one.

Proof

Similarly to DFS and BFS, after expanding a node x, A* has found paths from s to all nodes directly reachable via x. The only case where A* stops without finding a path to t is when no further reachable unexpanded nodes exist. This implies no path to t exists in the graph.

2.3.1 | Heuristics and Optimality

A* is not unconditionally guaranteed to find an optimal path. Its optimality depends entirely on the properties of the chosen heuristic $h(x)$.

Def. 12 **Admissible Heuristic**

Let $h^*(x)$ be the true minimal cost from x to t. A heuristic $h(x)$ is **admissible** if it represents a lower bound. Formally:

$$h(x) \leq h^*(x) \quad \forall x \in V$$

An admissible heuristic is always "optimistic" (it never overestimates the cost). $d_1(x,t)$, $d_2(x,t)$, and even $h(x) = 0$ are all admissible heuristics.

(it can be optimistic but not pessimistic because if it is optimistic you might just end up taking more time, but if it is pessimistic you might miss the shortest path (a pessimistic heuristic overestimates the cost, which can cause the algorithm to falsely assume the optimal path is too expensive and settle for a suboptimal route))

Thm. 3 **Optimality**

If $h(x)$ is admissible, A* is guaranteed to find a least-cost path.

Proof

Suppose A* finds a suboptimal path with cost $f' > f^* = h^*(s)$. There must exist a node x which is unexpanded and whose path from s is the beginning of a true shortest path to t. Since A* terminated, it must be that $f(x) \geq f'$. However, we know: $f(x) = g(x) + h(x) = g^*(x) + h(x) \leq g^*(x) + h^*(x) = f^*$ So, $f^* \geq f(x) \geq f'$, which contradicts the assumption that $f' > f^*$.

Def. 13 **Monotonic (Consistent) Heuristic**

A heuristic $h(x)$ is **monotonic** if it satisfies the triangle inequality for all edges $(x,y) \in A$, and the goal heuristic is zero:

$$h(x) \leq w_{xy} + h(y) \quad \text{and} \quad h(t) = 0$$

- monotonic heuristics ensure that the f-values along any path are non-decreasing ($f(x) \leq f(y)$)
- it is impossible to decrease $f(x)$ by extending a path
- this guarantees no re-expansions of nodes are ever necessary, making it comparable to Dijkstra (on non-negative weights)

> **N.B.**: monotonicity implies admissibility!

2.4 | Complexity and Variants

A* is optimally efficient with respect to the used heuristic $h(x)$, meaning no other complete algorithm using the exact same heuristic will expand fewer nodes.

- note that Dijkstra's algorithm is the special case of A* where $h(x) = 0$!

The runtime is generally $O(d^l)$ (where l is the number of nodes on the shortest path and d is a constant), but the actual performance relies heavily on the quality of $h(x)$.

> Heuristic Dominance and the 8-Puzzle
>
> A better heuristic leads to better A* performance. If $h_1(x) \leq h_2(x)$ for all $x \in V$, we say that h_2 **dominates** h_1. A* is guaranteed to expand no more nodes using h_2 than it would with h_1.
>
> With a nearly perfect heuristic $h(x) \approx h^*(x)$, runtime drops to $\Theta(l)$.
>
> For example, for the 8-Puzzle problem:
>
> - $h_1(x)$: number of tiles in wrong positions.
> - $h_2(x)$: sum of Manhattan distances of each tile to its target location.
>
> Because h_2 dominates h_1, running A* with h_2 reduces the number of expanded nodes compared to h_1 (e.g., 1,641 nodes vs 39,135 nodes for a 24-move solution).

2.4.1 | Improvements and Variants

For very complex problems, A* might still consume too much memory or time. Common improvements include:

- **Bidirectional search**: running search from s to t and from t to s simultaneously until they meet.
- **Weighted A***: "inflating" the heuristic (e.g., $f(x) = g(x) + W \cdot h(x)$) to place more emphasis on reaching the goal quickly.

  this sacrifices admissibility (optimality is no longer guaranteed), but it provides a bounded sub-optimality in exchange for big speedups
- **Preprocessing**: pre-calculating all-pairs shortest paths or storing pairwise minimum distances to create near-perfect heuristics
- **Hierarchical approach**: searching on abstraction levels (e.g., plan a route on major highways first, then refine to local main roads, and finally down to street level)
- **Iterated Deepening A* (IDA*)**: designed primarily to limit memory usage; it mimics Iterative Deepening DFS, performing a depth-first search while pruning any node where $f(x)$ exceeds a progressively increasing threshold c.

  while it guarantees finding the optimal solution with lower memory bounds, it is generally much slower than standard A* due to repeated work

CHAPTER 3

Planar Graphs

3.1 | Basic Concept

A graph $G = (V, E)$ is called planar if it can be drawn in the plane in a way such that no two edges intersect geometrically except at a vertex to which both are incident. Such a drawing is called a **planar embedding** of G, or a plane graph. A plane graph G divides the plane into connected regions called **faces**. The unbounded region is called the outer or external face of G.

3.2 | Euler's Formula

Thm. 4 **Euler's formula**

Let $G = (V, E)$ be a connected, planar graph, and let f denote the number of faces. Then

$$|V| - |E| + f = 2$$

holds for any planar embedding of G.

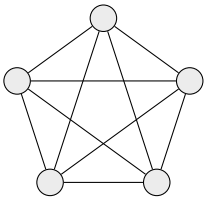
From Euler's formula, we can derive several important corollaries:

- if G is a simple, planar graph with $|V| \geq 3$, then the number of edges is bounded by $|E| \leq 3|V| - 6$
- for any simple, planar, bipartite graph with $|V| \geq 3$ vertices and $|E|$ edges, the bound is tighter: $|E| \leq 2|V| - 4$
- each simple, planar graph contains a vertex v of degree $d(v) \leq 5$

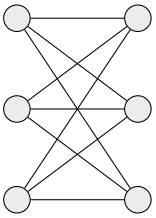
3.3 | Characterizations of Planar Graphs

Certain graphs cannot be drawn in the plane without crossing edges. The two fundamental examples are known as the Kuratowski graphs:

K_5



$K_{3,3}$



- K_5: the complete graph on 5 vertices

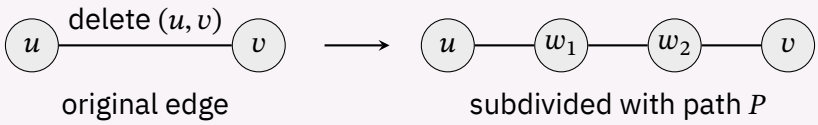
- $K_{3,3}$: the complete bipartite graph with two partitions of 3 vertices each

To establish whether a graph is planar, we define relationships to these Kuratowski graphs using subdivisions and minors.

Def. 14 **Subdivision**

Subdividing an edge $(u, v) \in E$ of a graph $G = (V, E)$ is the operation of deleting (u, v) and adding a path $P = (u, w_1, ..., w_k, v)$, $w_i \notin V$, $1 \leq i \leq k$ to G.

A graph G is called a subdivision of a graph G' if it is obtained by subdividing some of the edges of G'.

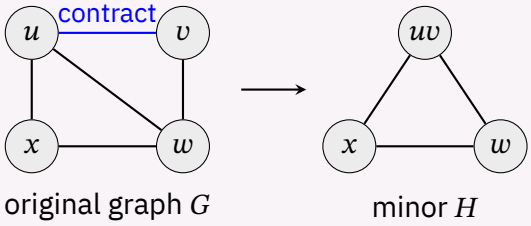


Thm. 5 **[Kuratowski, 1930]**

A graph is planar if and only if it does not contain a subdivision of K_5 or $K_{3,3}$.

Def. 15 **Graph Minor**

A graph H is a minor of G if H is obtained from G by deleting and contracting edges and/or deleting vertices.

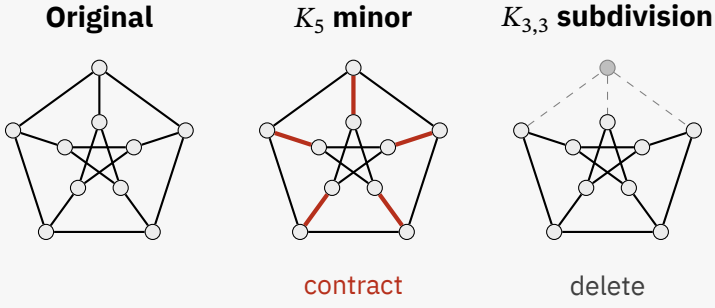


Thm. 6 **[Wagner]**

A graph is planar if and only if it has no minor isomorphic to K_5 or $K_{3,3}$.

Example: Peterson graph

The Peterson graph is a classic example of a non-planar graph.



- by contracting specific edges, we obtain the K_5 minor
- by deleting specific edges, we obtain a subdivision of $K_{3,3}$

Thus, we can prove its non-planarity with both Wagner's and Kuratowski's theorems.

3.4 | Linear Planarity Testing Algorithms

Several algorithmic approaches have been developed to test planarity and construct planar embeddings in linear time:

- **Path-Addition Approach**: the first linear-time planarity testing algorithm, introduced by Hopcroft and Tarjan (1974).

  starts from a cycle and adds one path at a time to the structure

  (Mehlhorn and Mutzel (1996) later clarified how to use this to construct the actual planar embedding)

- **Vertex-Addition Approach**: first considered by Lempel, Even, and Cederbaum (1967)

  adds vertices one-by-one following an st-numbering

  (Booth and Lueker (1976) proved it can be implemented in linear time using PQ-trees, and Chiba et al. (1985) showed how to construct the embedding)

- **Block Embedding Approach**: proposed by Shih and Hsu (1999) and Boyer and Myrvold (1999)

  first constructs a DFS-tree and then adds back edges to iteratively construct a planar embedding if one exists

  Boyer and Myrvold (2004) later introduced a new variant with a simpler implementation based on edge addition